

Capstone Project: **Modular** **Image Processing Application**

Vidhan Jaikumar Shivani (3121420)

1.Introduction

- The module is fully implemented using **pure NumPy**, without relying on OpenCV, scikit-image, SciPy, or any external image-processing libraries. It contains **11 image-processing techniques**, designed to demonstrate both foundational and advanced concepts in digital image enhancement.

Implemented Transformations

1. **Color Inversion (Negative)**
2. **Brightness Adjustment**
3. **Binary Thresholding**
4. **Power Law Transformation (Gamma Correction)**
5. **Color Profiles (Multi-Effect Suite)**
 - Sepia Tone
 - Cyanotype (Blueprint)
 - Monochrome (Grayscale)
 - RGB Channel Swap
 - Night Vision (Green Phosphor)
 - Autumn Warmth (Golden Hour)
 - Red Channel Isolation
 - Green Channel Isolation
 - Blue Channel Isolation
6. **Solarization**
7. **Vignette Effect**
8. **Box Blur (3×3 Smoothing Filter)**
9. **Sharpening (Laplacian Kernel) Custom Convolution (User-Defined 3×3 Kernel)**
10. **Canny Edge Detection (Multi-Stage Advanced Algorithm)**

The module follows the course theme of *“the more, the merrier”* by including a diverse set of transformations, with the **Color Profiles** section offering multiple sub-techniques for creative and stylistic image manipulation.

User Experience Enhancements

Why UX mattered in my module

Image processing can freeze the interface during heavy computations, which makes the app feel unresponsive and makes debugging harder. To avoid that “stuck” feeling, I added several small but meaningful UX improvements that make the module smoother, clearer, and more professional to use.

Key UX Features I Implemented:

- **Forced UI Refresh** (`QApplication.processEvents()`) : Ensures the interface updates before the heavy NumPy operations begin. This prevents the window from freezing and gives immediate visual feedback.
- **Indeterminate Progress Bar** : Shows a continuous loading animation while processing is happening. Even if the operation takes time, the user sees that the system is actively working.
- **State-Managed Apply Button** : The “Apply Processing” button is temporarily disabled during execution. This prevents accidental double-clicks and avoids overlapping operations.
- **Clear Operation Descriptions** : Each filter displays a short explanation so users understand what they’re applying.
- **Dynamic Parameter Widgets** : The UI automatically switches to the correct parameter panel based on the selected operation, keeping the interface clean and intuitive.

Select Module:

Vidhan_Module

Vidhan_Module

Select Filter:

11. Canny Edge Detection

Multi-stage edge detector using Gaussian smoothing, Sobel gradients, non-max suppression, and hysteresis.

Low Threshold (Min: 0 | Max: 255):

50

High Threshold (Min: 0 | Max: 255):

120

Processing... Please wait.

Foundational Image Processing Techniques

1. Color Inversion (Negative)

A simple but essential point operation that flips the brightness of every pixel.

- **What it does:** Turns light areas dark and dark areas light.
- **Why it matters:** Useful for understanding pixel-wise operations and intensity transformations.

Formula: $\text{output} = 255 - \text{input}$

Foundational Image Processing Techniques

Modular Image Viewer (Python Native)

— □ ×

Load Image...

Select Module:

Vidhan_Module

Vidhan_Module

Select Filter:

1. Color Inversion (Negative)

Inverts pixel colors.

No parameters needed. Just click Apply!

Apply Processing

Original Image



Processed Image



Foundational Image Processing Techniques

2. Brightness Adjustment

Adds or subtracts a constant value from every pixel.

- **What it does:** Lightens or darkens the entire image evenly.
- **How it works:** $\text{output} = \text{input} + c$
- **Implementation detail:** Uses int16 internally to avoid overflow, then clips back to 0–255.

Foundational Image Processing Techniques

Modular Image Viewer (Python Native)

Load Image...

Select Module:
Vidhan_Module

Vidhan_Module

Select Filter:
2. Brightness Adjustment

Evenly lightens or darkens the entire image.

Brightness Intensity (Min: -255 | Max: 255):
125,00

Apply Processing

Original Image



Processed Image



Modular Image Viewer (Python Native)

Load Image...

Select Module:
Vidhan_Module

Vidhan_Module

Select Filter:
2. Brightness Adjustment

Evenly lightens or darkens the entire image.

Brightness Intensity (Min: -255 | Max: 255):
-125,00

Apply Processing

Original Image



Processed Image



Foundational Image Processing Techniques

3. Binary Thresholding

Converts an image into pure black and white based on a threshold.

- **What it does:** Segments the image into two intensity regions.
- Luminosity Method is used to arrive on the conclusion for grayscale conversion.

- **Grayscale conversion:**

$$0.299R + 0.587G + 0.114B$$

- **Threshold rule:**

$$\text{output} = \begin{cases} 255 & \text{if gray} > T \\ 0 & \text{otherwise} \end{cases}$$

Foundational Image Processing Techniques

Modular Image Viewer (Python Native)

— □ ×

Load Image...

Select Module:

Vidhan_Module

Vidhan_Module

Select Filter:

3. Binary Thresholding

Converts the image to pure black and white based on a threshold limit.

Level (Min: 0 | Max: 255):

127

Apply Processing

Original Image



Processed Image



Foundational Image Processing Techniques

4. Gamma Correction (Power Law Transformation)

Why Gamma Matters

Many images look washed-out or too dark because the relationship between pixel intensity and perceived brightness is non-linear.

Gamma correction fixes this by redistributing mid-tones without destroying highlights or shadows.

What Gamma Correction Does

- Brightens or darkens **mid-tones** specifically
- Preserves overall contrast better than simple brightness adjustment
- Used in photography, displays, and image enhancement pipelines

My Optimization: High-Speed LUT

- Instead of computing the power function for every pixel (slow), I precompute a **256-value lookup table**:

Why This Is Important

- LUT makes gamma correction **extremely fast**
- Reduces CPU load during presentations or large images
- Demonstrates understanding of both **image theory** and **performance engineering**

Mathematical Model

$$\text{output} = 255 \cdot \left(\frac{\text{input}}{255} \right)^\gamma$$

- $\gamma < 1 \rightarrow$ brightens the image
- $\gamma > 1 \rightarrow$ darkens the image

$$\text{LUT}[i] = 255 \cdot \left(\frac{i}{255} \right)^\gamma$$

Foundational Image Processing Techniques

Modular Image Viewer (Python Native)

Load Image...

Select Module:
Vidhan_Module

Vidhan_Module

Select Filter:
4. Power Law (Gamma)

Non-linearly adjusts mid-tones to fix washed-out images using a high-speed LUT.

Gamma (Min: 0.01 | Max: 5.0):
3,00

Apply Processing

Original Image



Processed Image



Modular Image Viewer (Python Native)

Load Image...

Select Module:
Vidhan_Module

Vidhan_Module

Select Filter:
4. Power Law (Gamma)

Non-linearly adjusts mid-tones to fix washed-out images using a high-speed LUT.

Gamma (Min: 0.01 | Max: 5.0):
0,10

Apply Processing

Original Image



Processed Image



Foundational Image Processing Techniques

Color Profiles Suite (Multi-Effect Color Processing)

Why Color Profiles Matter ?

Color transformations showcase creative image manipulation using pure NumPy. This module features 9 custom color effects demonstrating matrix operations, channel manipulation, and tone mapping.

What This Section Includes

- Sepia Tone: Warm, vintage look via a 3x3 matrix.
- Cyanotype (Blueprint): Blue-tinted monochrome style.
- Monochrome (Grayscale): Perceptually accurate grayscale using luminance weights.
- RGB Channel Swap: Reorders channels (RGB → BGR) for surreal shifts.
- Night Vision (Green Phosphor): Grayscale converted and green channel amplified.
- Autumn Warmth (Golden Hour): Boosts reds, reduces blues for sunset light.
- Channel Isolation: Red, Green, or Blue channels isolated.

Implementation Details

- Sepia: Matrix multiplication.
- Monochrome: Weighted grayscale conversion.
- Night Vision/Autumn Warmth: Channel-wise scaling.
- Isolation Effects: Direct channel manipulation.
- All effects use pure NumPy.

Foundational Image Processing Techniques

Modular Image Viewer (Python Native)

Load Image...

Select Module:
Vidhan_Module

Vidhan_Module

Select Filter:
5. Color Profiles (Tones & RGB)

Apply matrix transformations to shift colors to Sepia, Cyanotype, Grayscale, or isolated channels.

Select Tone/Channel:
Sepia Tone

Apply Processing

Original Image

Processed Image

Modular Image Viewer (Python Native)

Load Image...

Select Module:
Vidhan_Module

Vidhan_Module

Select Filter:
5. Color Profiles (Tones & RGB)

Apply matrix transformations to shift colors to Sepia, Cyanotype, Grayscale, or isolated channels.

Select Tone/Channel:
Monochrome (Grayscale)

Apply Processing

Original Image

Processed Image

Modular Image Viewer (Python Native)

Load Image...

Select Module:
Vidhan_Module

Vidhan_Module

Select Filter:
5. Color Profiles (Tones & RGB)

Apply matrix transformations to shift colors to Sepia, Cyanotype, Grayscale, or isolated channels.

Select Tone/Channel:
Cyanotype (Blueprint)

Apply Processing

Original Image

Processed Image

Modular Image Viewer (Python Native)

Load Image...

Select Module:
Vidhan_Module

Vidhan_Module

Select Filter:
5. Color Profiles (Tones & RGB)

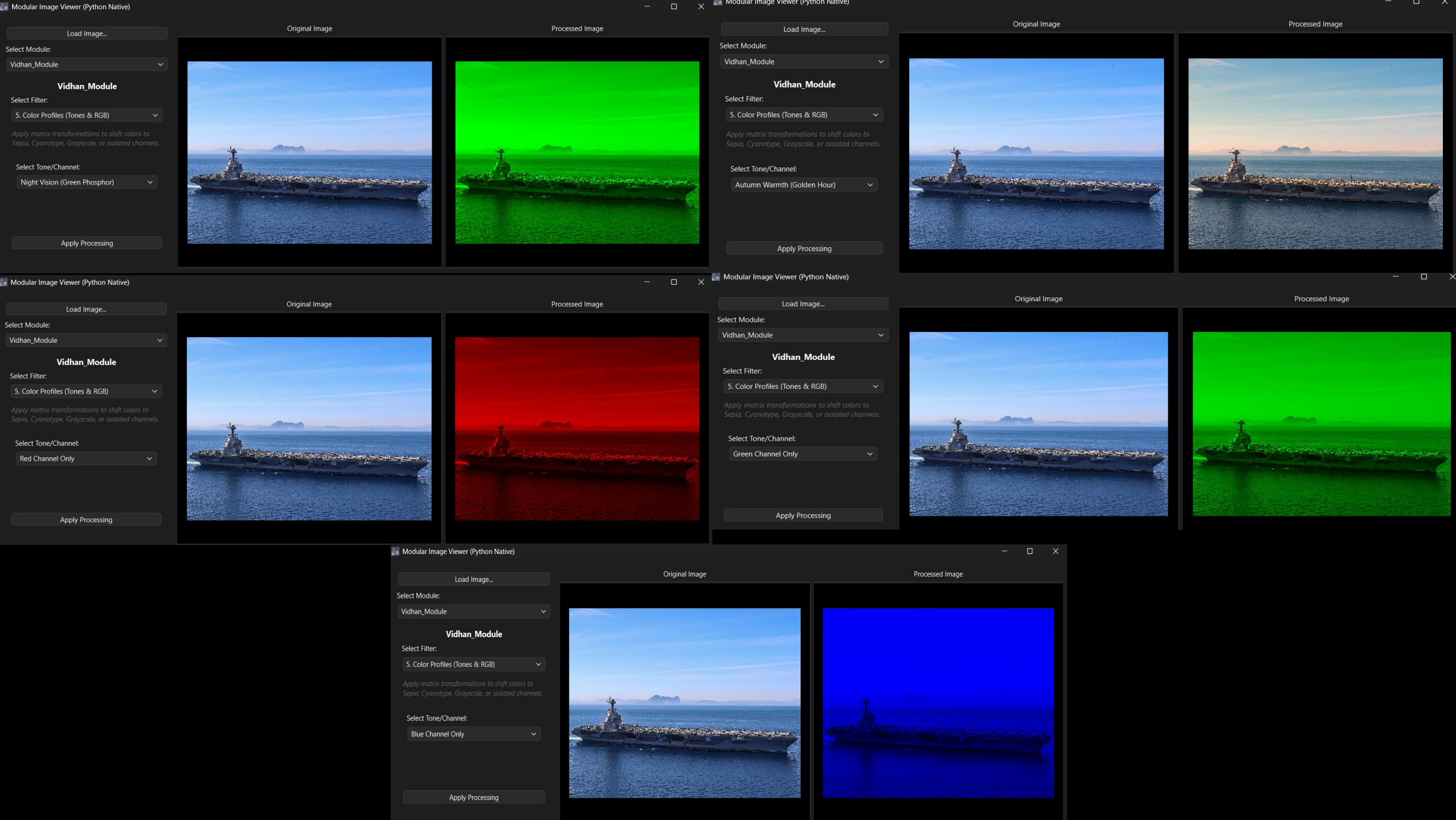
Apply matrix transformations to shift colors to Sepia, Cyanotype, Grayscale, or isolated channels.

Select Tone/Channel:
Color Swap (RGB to BGR)

Apply Processing

Original Image

Processed Image



Foundational Image Processing Techniques

Solarization (Darkroom-Inspired Effect)

What Solarization Does

Solarization is a classic photographic effect where only the **brightest parts** of the image get inverted.

It creates a surreal, high-contrast look inspired by traditional darkroom techniques.

How It Works

The image is split into two intensity regions using a threshold:

- **Pixels below the threshold** → stay the same
- **Pixels above the threshold** → get inverted

This gives a dramatic “partial negative” effect.

Why This Effect Is Interesting

- It blends **normal** and **inverted** regions in a single image
- Highlights bright areas dramatically
- Produces artistic, stylized results
- Shows how conditional pixel operations work in NumPy

Mathematical Rule

$$\text{output}(x, y) = \begin{cases} I(x, y) & \text{if } I(x, y) < T \\ 255 - I(x, y) & \text{if } I(x, y) \geq T \end{cases}$$

Where:

- $I(x, y)$ is the pixel intensity
- T is the threshold value

Foundational Image Processing Techniques

Modular Image Viewer (Python Native)

— □ ×

Load Image...

Select Module:

Vidhan_Module

Vidhan_Module

Select Filter:

6. Solarization

A darkroom effect that inverts only the pixels above a certain threshold.

Level (Min: 0 | Max: 255):

127

Apply Processing

Original Image

A photograph of a large blue ship, possibly a naval vessel, sailing on a calm blue sea under a clear blue sky. In the background, there are distant landmasses and mountains.

Processed Image

The same ship and sea scene as the original image, but processed with a solarization effect. The colors are inverted, resulting in a dark, high-contrast image where the ship and sea appear in shades of blue and black, while the sky is a bright, hazy white.

Foundational Image Processing Techniques

What the Vignette Effect Does

The vignette effect darkens the corners of an image while keeping the center bright.

It mimics the natural falloff seen in old lenses, film cameras, and cinematic grading.

How It Works

A smooth, circular gradient is generated across the image.

Pixels farther from the center receive a stronger darkening effect.

Mathematical Idea

A radial distance map is created using:

$$r(x, y) = \sqrt{x^2 + y^2}$$

Then the mask is computed as:

$$\text{mask}(x, y) = 1 - \text{clip}(r(x, y), 0, 1)$$

- Center of the image $\rightarrow$ mask ≈ 1 (no darkening)
- Corners $\rightarrow$ mask $\rightarrow 0$ (strong darkening)

Foundational Image Processing Techniques

Modular Image Viewer (Python Native)

— □ ×

Load Image...

Select Module:

Vidhan_Module

Vidhan_Module

Select Filter:

7. Vignette Effect

Darkens the corners using a mathematically generated radial gradient mask.

No parameters needed. Just click Apply!

Apply Processing

Original Image



Processed Image



Foundational Image Processing Techniques

Convolution Filters (Box Blur & Sharpening)

Why Convolution Matters

Convolution is one of the core building blocks of image processing.

It allows us to apply effects like smoothing, sharpening, edge detection, and custom filters by sliding a small matrix (kernel) over the image and combining pixel values.

1. Box Blur (3×3 Smoothing Filter)

Averaging the neighborhood reduces noise and softens details.

What it does:

- Smooths the image
- Reduces high-frequency noise
- Produces a soft, even blur

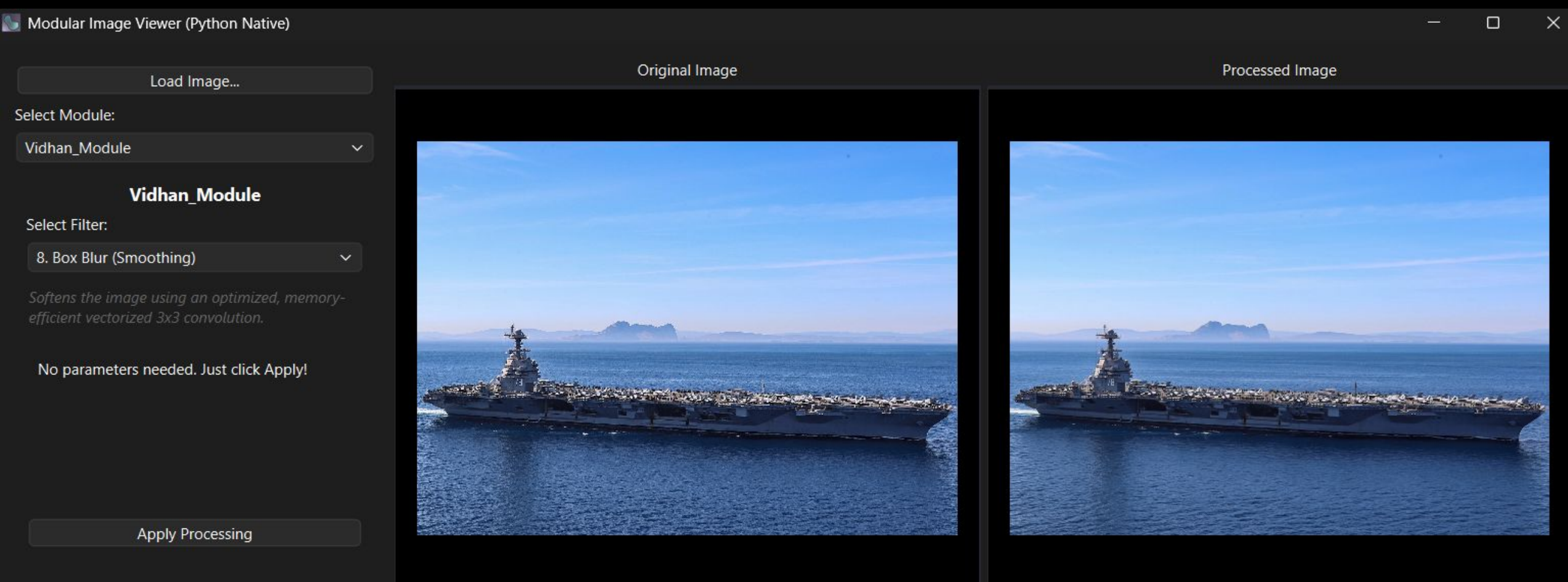
How I implemented it:

- Normalized 3×3 kernel
- Manual convolution using NumPy
- Channel-wise processing for RGB images

Kernel:

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Foundational Image Processing Techniques



Foundational Image Processing Techniques

2. Sharpening (Laplacian-Based Enhancement)

Sharpening boosts edges by emphasizing intensity differences. **What it does:**

- Enhances edges and fine details
- Makes textures pop
- Increases local contrast

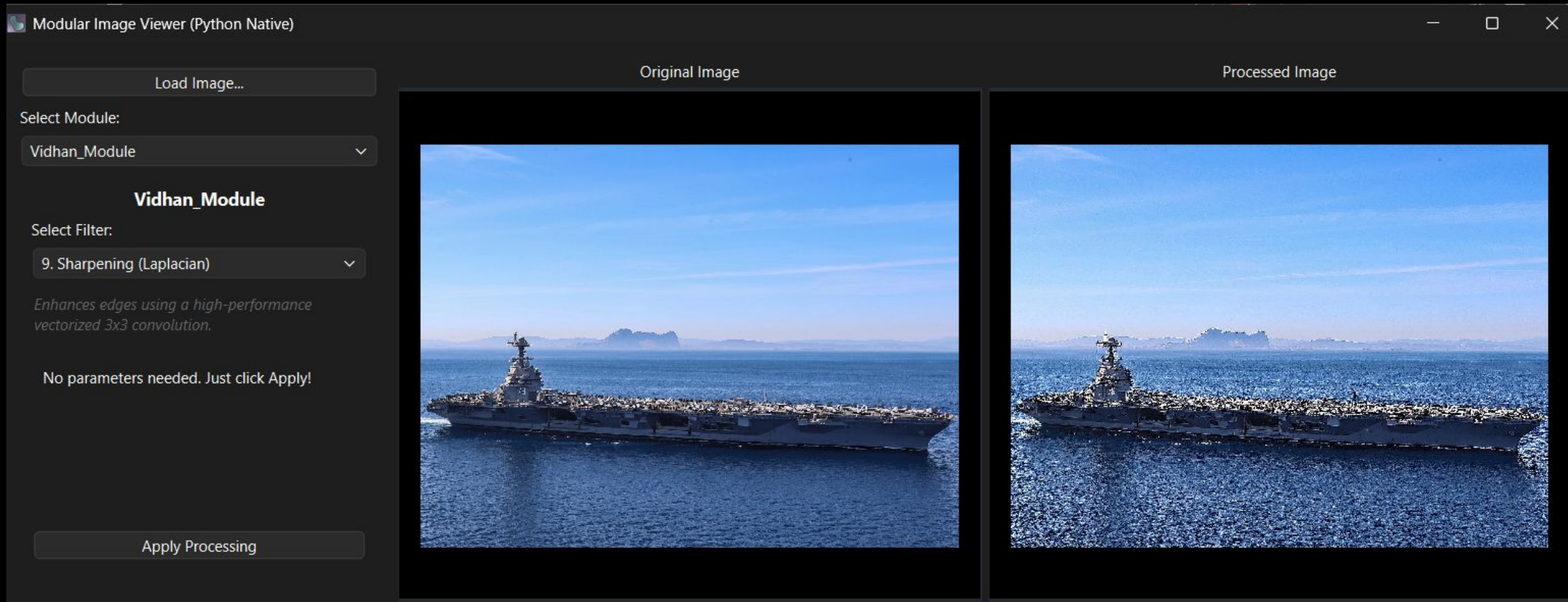
How it works:

- The center pixel is multiplied by 5
- Surrounding pixels are subtracted
- This highlights regions with strong intensity changes

Kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Foundational Image Processing Techniques



Foundational Image Processing Techniques

Custom Convolution (User-Defined 3×3 Kernel)

Why Custom Convolution Matters

Custom convolution is one of the most powerful features in image processing.

Instead of giving the user a fixed effect, this tool lets them **design their own filter** by entering a 3×3 kernel. This turns the module into a mini-image-processing lab.

What the User Can Do

- Create their own blur filters
- Design sharpening or embossing kernels
- Experiment with edge detectors
- Build artistic or experimental effects
- Understand how spatial filtering works at a low level

How Convolution Works

A 3×3 kernel slides across the image.

For each pixel, the output is:

$$\text{output}(i, j) = \sum_{m=-1}^1 \sum_{n=-1}^1 k[m, n] \cdot I[i + m, j + n]$$

Where:

- $k[m, n]$ is the kernel value
- $I[i + m, j + n]$ is the neighboring pixel
- The result is the weighted sum of the neighborhood

This is the foundation of **blurring**, **sharpening**, **edge detection**, **embossing**, and many other effects.

Foundational Image Processing Techniques

Modular Image Viewer (Python Native)

— □ ×

Load Image...

Select Module:

Vidhan_Module

Vidhan_Module

Select Filter:

10. Custom Convolution

Input your own 3x3 matrix to create custom filters!
Features Auto-Normalization.

Custom 3x3 Kernel (Auto-Normalized):

1,0	50,	100
50,	1,0	50,
100	50,	1,0

Apply Processing

Original Image



Processed Image



The 5 Stages of Canny Edge Detection



Canny Edge Detection (Multi-Stage Advanced Algorithm)

Why Canny Is Special

Canny is not just another filter — it is a **full pipeline** of image processing steps designed to produce clean, thin, noise-free edges.

It is widely used in computer vision, robotics, medical imaging, and industrial inspection.

Foundational Image Processing Techniques

Modular Image Viewer (Python Native)

Load Image...

Select Module:

Vidhan_Module

Vidhan_Module

Select Filter:

11. Canny Edge Detection

Multi-stage edge detector using Gaussian smoothing, Sobel gradients, non-max suppression, and hysteresis.

Low Threshold (Min: 0 | Max: 255):

50

High Threshold (Min: 0 | Max: 255):

120

Apply Processing

Original Image

A photograph of a large naval ship, possibly a destroyer or cruiser, sailing on a blue sea under a clear blue sky. The ship is viewed from a side-on perspective, moving towards the right.

Processed Image

The result of applying Canny edge detection to the original image. The ship and the horizon are highlighted as white lines on a black background, showing the detected edges of the vessel and the sea.

Github Pull Request

The screenshot shows the Visual Studio Code interface with a terminal window and a file explorer. The terminal window displays the output of a `git push` command, which includes a message to create a pull request on GitHub. The file explorer shows the project structure, including a `vidhan` directory containing `vidhan_module.py`.

Terminal Output:

```
PS C:\Users\VS\srh-image-viewer> git add src/modules/vidhan/vidhan_module.py
PS C:\Users\VS\srh-image-viewer> git commit -m "feat: Add Vidhan module with 11 custom image filters and UI"
[main 6506c7d] feat: Add Vidhan module with 11 custom image filters and UI
1 file changed, 462 insertions(+)
create mode 100644 src/modules/vidhan/vidhan_module.py
PS C:\Users\VS\srh-image-viewer> git push -u origin feature/vidhan
error: src refspec feature/vidhan does not match any
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 5.46 KiB | 1.36 MiB/s, done.
Total 6 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote:
remote: Create a pull request for 'feature/vidhan' on GitHub by visiting:
remote:   https://github.com/guijoe/srh-image-viewer/pull/new/feature/vidhan
remote:
To https://github.com/guijoe/srh-image-viewer.git
 * [new branch]      feature/vidhan -> feature/vidhan
branch 'feature/vidhan' set up to track 'origin/feature/vidhan'.
PS C:\Users\VS\srh-image-viewer>
```

File Explorer:

- assets
- src
 - __pycache__
 - modules
 - __pycache__
 - joel
 - sample
 - vidhan
 - vidhan_module.py
 - _init_.py
 - abdul_basit
 - i_image_module.py
 - ui_components
 - image_data_store.py
 - main_app.py
 - module_manager.py
 - README.md

Code Editor:

The code editor shows the `vidhan_module.py` file. The code includes imports for Qt widgets and core Qt modules, and defines a `NoParamsWidget` class.

```
1 from PySide6.QtWidgets import (QWidget, QVBoxLayout, QLabel, QPushButton,
2                               QComboBox, QStackedWidget, QDoubleSpinBox, QSpinBox,
3                               QGridLayout, QProgressBar, QApplication)
4 from PySide6.QtCore import Qt, Signal
5 import numpy as np
6 import imageio.v3 as iio
7 import time
8
9 from modules.i_image_module import IImageModule
10 from modules.sample.sample_module import BaseParamsWidget
11 # 1. THE UI WIDGETS (Parameter Inputs & UX Enhancements)
12 class NoParamsWidget(BaseParamsWidget):
13     def __init__(self, parent=None):
14         super().__init__(parent)
15         layout = QVBoxLayout(self)
16         layout.addWidget(QLabel("No parameters needed. Just click Apply!"))
17         layout.addStretch()
18         def get_params(self) -> dict: return {}
19
20 class BrightnessParamsWidget(BaseParamsWidget):
21     def __init__(self, parent=None):
22         super().__init__(parent)
23         layout = QVBoxLayout(self)
24         layout.addWidget(QLabel("Brightness Intensity (Min: -255 | Max: 255):"))
25         self.val_spin = QDoubleSpinBox()
26         self.val_spin.setRange(-255.0, 255.0)
```

Thank You
